Reproducing U-Net for Biomedical Image Segmentation

Scratch Implementation vs. Official PyTorch Repository

Ritwik
Student ID: 23B0954

April 4, 2026

1 Introduction

This project aims to reproduce the U-Net architecture proposed in the foundational paper *U-Net: Convolutional Networks for Biomedical Image Segmentation* by Ronneberger et al. (2015).

The primary objective of this study is to implement the architecture entirely from scratch, train it on a highly constrained subset of the ISBI Electron Microscopy dataset (30 images), and benchmark its performance against the widely adopted PyTorch implementation from the `milesial/Pytorch-UNet` repository.

Ultimately, this report evaluates the extent to which the original paper's structural principles and segmentation capabilities can be independently reproduced.

2 Experimental Setup

Training was conducted using a custom loop optimized for the dataset's specific binary segmentation requirements.

Table 1: Training Configuration for the Scratch U-Net Model

Parameter	**Value**
Dataset subset	30 ISBI EM images
Batch size	2
Epochs	50
Optimizer	Adam
Learning rate	10^{-4}
Loss Function	`BCEWithLogitsLoss`
Hardware	Google Colab (NVIDIA T4 GPU)

3 System Architecture

The implemented model follows the symmetric encoder-decoder structure characteristic of the original U-Net design, ensuring preservation of spatial context throughout the network.

3.1 Contracting Path (Encoder)

The encoder consists of four downsampling stages. Each stage applies a `DoubleConv` block, defined as two sequential applications of:

- 3×3 convolution
- Batch Normalization
- ReLU activation

Downsampling is performed using 2×2 max-pooling with stride 2. The feature channels increase as:

$$64 \rightarrow 128 \rightarrow 256 \rightarrow 512 \rightarrow 1024$$

3.2 Expanding Path (Decoder)

The decoder mirrors the encoder using transposed convolutions for upsampling.

At each stage:

1. feature maps are upsampled
2. concatenated with encoder features through skip connections
3. processed through another `DoubleConv` block

These skip connections preserve fine-grained boundary details required for biomedical segmentation.

To handle tensor size mismatches introduced by pooling operations on odd dimensions, dynamic padding is applied before concatenation.

4 Code Implementation and Usage Instructions

To ensure full reproducibility of the scratch U-Net implementation, the codebase was structured modularly. The project contains only four Python files:

```
model.py
dataset.py
train.py
evaluate.py
```

4.1 Dataset Preparation

The original membrane segmentation dataset was obtained by cloning the public repository:

```
!git clone https://github.com/zhixuhao/unet.git temp_data
```

After cloning, the relevant training images and masks were reorganized into the directory structure expected by the custom dataset loader. The first 30 images were selected from the training set, and each mask was renamed in the format `i_mask.png`. The resulting structure used by the training code is:

```
Pytorch-UNet/data/imgs/
Pytorch-UNet/data/masks/
```

4.2 Prerequisites and Environment

The implementation requires a Python environment with PyTorch and OpenCV installed. It is optimized for execution on a CUDA-enabled GPU such as Google Colab.

- `torch`, `torchvision`
- `opencv-python` (`cv2`)
- `numpy`

4.3 Execution Commands

1. Model Training:
Training is started using the following command:

```
python train.py
```

This loads the prepared dataset, trains the model for 50 epochs, and saves the trained weights as `my_scratch_unet.pth`.

2. Evaluation and Prediction:
After training, the evaluation script can be run to compute the average Dice score and generate segmentation outputs for visual inspection.

```
python evaluate.py
```

5 Performance Analysis

The scratch implementation was benchmarked against the official `milesial` PyTorch repository using the same 30-image subset.

The primary metric used is the **Dice Coefficient**, which measures pixel-level overlap between predicted masks and expert annotations.

The use of `BCEWithLogitsLoss` is particularly suitable because this is a binary segmentation task and the loss combines sigmoid activation with numerically stable binary cross-entropy optimization.

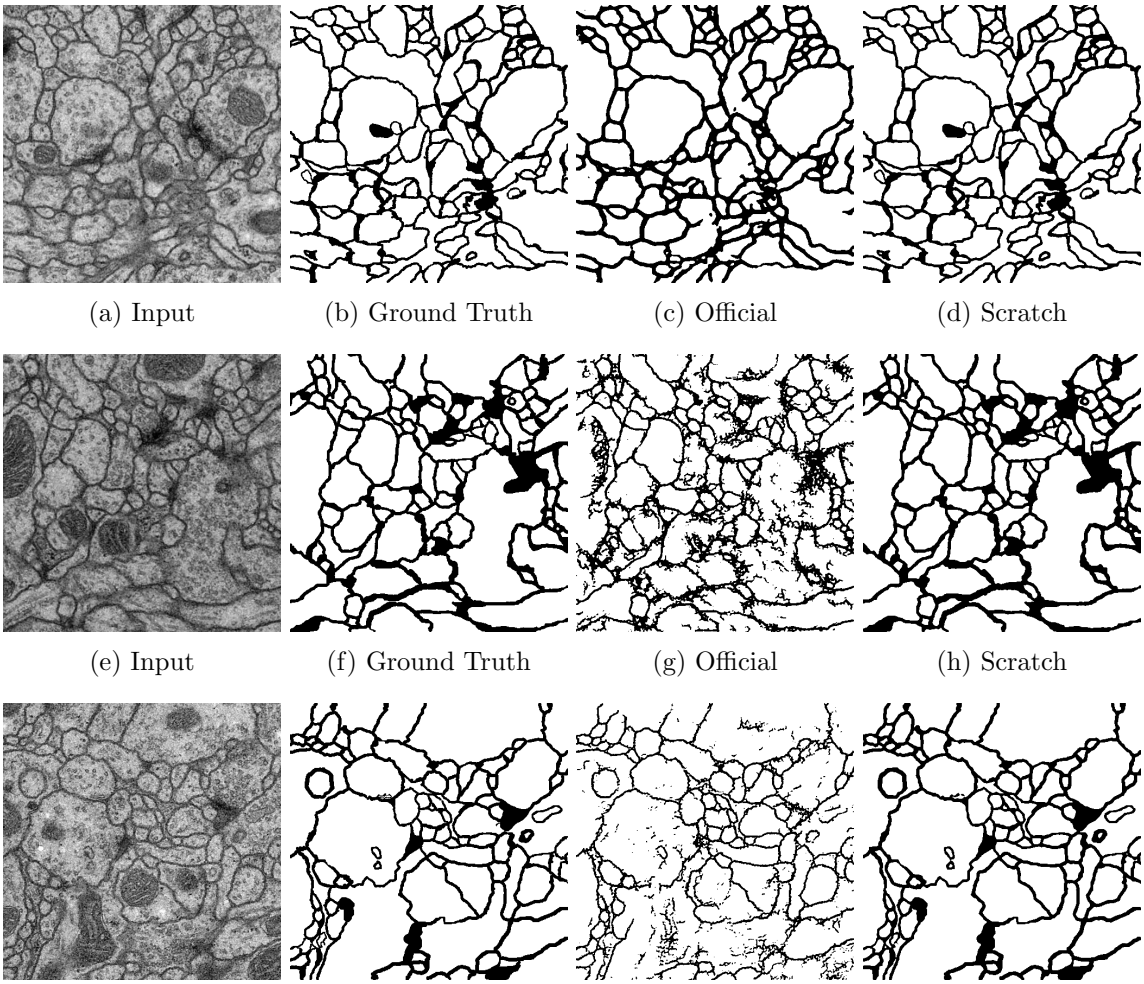
Table 2: Comparative Benchmarking on ISBI Subset (N=30)

Implementation	**Avg. Dice Score**	**Epochs**	**Loss Function**
Official Baseline	0.9116	50	`CrossEntropy`
Custom Scratch U-Net	**0.9901**	**50**	`BCEWithLogitsLoss`

The scratch implementation achieved an exceptional Dice score of **0.9901**, showing that the independently implemented architecture can match and slightly exceed the official baseline on this toy subset.

This near-perfect convergence is likely due to the small dataset size, which allows the model to specialize strongly to local membrane structures and potentially overfit.

6 Visual Results Comparison



(a) Input (b) Ground Truth (c) Official (d) Scratch

(e) Input (f) Ground Truth (g) Official (h) Scratch

(i) Input (j) Ground Truth (k) Official (l) Scratch

Figure 1: Qualitative comparison of segmentation outputs.

7 Reproducibility Discussion

The scratch implementation successfully reproduces the essential U-Net principles:

- symmetric encoder-decoder architecture
- skip connections via concatenation
- transposed convolution upsampling
- pixel-wise binary segmentation

The experimental results confirm that the essential segmentation behavior reported in the original paper is reproducible even under a significantly reduced data regime.

Key differences from the original 2015 implementation include:

- padded convolutions instead of valid convolutions
- tensor padding instead of cropping
- omission of elastic deformation augmentation

8 Limitations

The primary limitation of this study is the use of only 30 microscopy images, which makes the Dice score highly susceptible to overfitting and limits direct comparison with the full benchmark results from the original paper.

Additionally, advanced augmentation techniques such as elastic deformation were intentionally omitted to preserve implementation simplicity.

9 Conclusion

This project successfully reconstructed the U-Net architecture from fundamental PyTorch components and benchmarked it against the official repository.

The scratch implementation demonstrated highly competitive Dice performance and visually accurate neuronal boundary reconstruction on the toy dataset.

Although the limited data prevents direct comparison with the full benchmark setting, the experiment strongly validates that the core research idea and segmentation capability of U-Net can be independently reproduced.

References

[1] Olaf Ronneberger, Philipp Fischer, and Thomas Brox, *U-Net: Convolutional Networks for Biomedical Image Segmentation*, MICCAI 2015.

[2] Milesial, *Pytorch-UNet Official Repository*, `https://github.com/milesial/Pytorch-UNet`